TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

Tài liệu tự học buổi 11 — Đóng gói và cấu hình theo từng stack

Docker cho ứng dụng web nhiều tầng [SE] · Nối pipeline → kho dữ liệu → dashboard và chạy theo lịch [DA] · Đóng gói mô hình thành REST API bằng FastAPI [AI] · Bug Log, logging và xử lý lỗi xuyên lớp [chung]. Mỗi phần có tệp cấu hình hoàn chỉnh trên bộ dữ liệu Mekong Mobile và phép thử tự kiểm.

Sản phẩm của tài liệu này là điều kiện để nộp Bài tập 2 (thành phần 4 — hệ thống tích hợp chạy được) và là nền cho gói đóng gói của Bài tập 3 (chạy được bằng ≤ 2 lệnh). Làm trong tuần của buổi 11; giảng viên kiểm ở workshop buổi 12.

Biên soạn: **TS. Nguyễn Trí Hải** · Bộ môn Kỹ thuật phần mềm

Tài liệu lưu hành nội bộ · Bộ tài liệu tự học phần chuyên ngành (30% thời lượng lý thuyết buổi 11). Tài liệu song ngữ: bản tiếng Việt ở nửa đầu, bản tiếng Anh (English version) ở nửa sau.

Cách dùng tài liệu này

Giảng viên giảng trên lớp (buổi 11)	Sinh viên tự đọc trong tài liệu này
Ba nguồn khác biệt giữa máy em và máy người khác; khoá phiên bản theo stack; bốn bước gỡ lỗi có hệ thống; Bug Log; bốn mức log, ba thứ mọi dòng log phải có và thứ tuyệt đối không được ghi; xử lý lỗi xuyên lớp; image, container, volume; Dockerfile tối thiểu; docker-compose cho ứng dụng kèm cơ sở dữ liệu.	Cấu hình đóng gói cụ thể cho stack của track mình: nhiều dịch vụ và thứ tự khởi động [SE]; nối ba pha và chạy theo lịch [DA]; đóng gói mô hình thành dịch vụ dự đoán [AI]. Phần D dùng chung: mẫu Bug Log, cấu hình logging cho Python, bộ xử lý lỗi duy nhất cho API Python.

Bốn điều đúng cho cả ba track — đọc trước khi vào phần của mình

1. MỌI CẤU HÌNH RA BIẾN MÔI TRƯỜNG. Không có chuỗi kết nối, mật khẩu, khoá API, hay đường dẫn tuyệt đối nào nằm trong mã nguồn hoặc trong tệp cấu hình được commit. Có .env.example liệt kê đủ mọi biến với giá trị mẫu; .env thật nằm trong .gitignore.

2. THỨ TỰ KHỞI ĐỘNG PHẢI ĐƯỢC ĐỊNH NGHĨA. Ứng dụng khởi động trước khi cơ sở dữ liệu sẵn sàng là lỗi khiến gói đóng gói của em chạy trên máy em nhưng không chạy trên máy giảng viên. Dùng healthcheck kèm depends_on, hoặc thêm vòng chờ kết nối khi khởi động.

3. LOG RA THIẾT BỊ XUẤT CHUẨN, KHÔNG RA TỆP TRONG CONTAINER. Tệp log ghi bên trong container sẽ mất khi container bị xoá. Ghi ra stdout để "docker compose logs" đọc được — đây là quy ước chuẩn của mọi hệ thống chạy container.

4. PHÉP THỬ CUỐI CÙNG LUÔN GIỐNG NHAU: thư mục trống · clone repo · làm đúng theo README · hệ thống lên được. Giảng viên làm phép thử này ở buổi 12 (BT2) và buổi 16 (BT3) — hãy tự làm ngay từ tuần này.

Ba lỗi cấu hình chiếm phần lớn số ca "trên máy em chạy được"

Lỗi	Biểu hiện trên máy người khác	Cách phát hiện sớm	Cách sửa
Đường dẫn tuyệt đối trong mã	Lỗi không tìm thấy tệp hoặc thư mục ngay khi khởi động	grep -rnE "/home/|/Users/|C:\\" src/ trong toàn bộ mã nguồn	Dùng đường dẫn tương đối tính từ thư mục gốc dự án, hoặc đưa vào biến môi trường
Cổng ghi cứng và bị chiếm	Lỗi "address already in use", hoặc dịch vụ lên nhưng gọi vào cổng khác	Chạy thử khi máy đang có một dịch vụ khác dùng cùng cổng	Đưa cổng ra biến môi trường; ghi trong README cách đổi khi cổng bận
Phụ thuộc cài sẵn trên máy mình	Lỗi thiếu thư viện, hoặc chạy sai vì phiên bản khác	Chạy trong container hoặc trong môi trường ảo mới hoàn toàn	Ghi phiên bản chính xác vào tệp khoá phiên bản và commit tệp đó

PHẦN A — Track SE: Đóng gói ứng dụng web nhiều tầng

A1. Dockerfile nhiều giai đoạn cho backend

Dockerfile nhiều giai đoạn (multi-stage) tách bước cài đặt và bước chạy. Kết quả: ảnh chạy nhỏ hơn, không chứa công cụ biên dịch, và không chứa dữ liệu nhạy cảm còn lại từ bước cài đặt.

backend/Dockerfile — ví dụ Node.js

```
# ---------- GIAI ĐOẠN 1: cài đặt phụ thuộc ----------
FROM node:20-alpine AS deps
WORKDIR /app
# Chép RIÊNG tệp khoá phiên bản trước, để Docker dùng lại lớp cache
# khi mã nguồn đổi mà phụ thuộc không đổi
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# ---------- GIAI ĐOẠN 2: ảnh chạy ----------
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production

# Chạy bằng người dùng không đặc quyền - yêu cầu bảo mật cơ bản
RUN addgroup -S app && adduser -S app -G app

COPY --from=deps /app/node_modules ./node_modules
COPY src ./src
COPY package.json ./

USER app
EXPOSE 8000

# Healthcheck để docker-compose biết dịch vụ đã sẵn sàng chưa
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s --retries=6 \
  CMD node -e
"fetch('http://localhost:8000/health').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"

CMD ["node", "src/server.js"]
```

Bốn điều Dockerfile của em phải có để đạt Bài tập 3

Ghim phiên bản ảnh nền cụ thể (node:20-alpine), không dùng thẻ "latest" — "latest" hôm nay và "latest" tháng sau là hai thứ khác nhau, phá nguyên tắc TÁI LẬP ĐƯỢC.

Chép tệp khoá phiên bản trước rồi mới chép mã nguồn — thứ tự này khiến việc dựng lại nhanh gấp nhiều lần.

Chạy bằng người dùng không đặc quyền (không chạy bằng root) — đây là một mục trong checklist bảo mật của buổi 15.

Có HEALTHCHECK — đây là thứ cho phép docker-compose biết khi nào dịch vụ đã thật sự sẵn sàng, thay vì chỉ biết container đã khởi động.

A2. docker-compose.yml đầy đủ — ba dịch vụ và thứ tự khởi động

docker-compose.yml — sao chép được, chỉ cần đổi tên dự án

```
services:

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB:       ${DB_NAME}
      POSTGRES_USER:     ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
      # Tệp .sql/.sh nằm TRỰC TIẾP trong db/init chạy TỰ ĐỘNG khi volume còn trống,
      # theo thứ tự tên: 01_schema.sql → 02_seed.sql. Thư mục con bị BỎ QUA.
      - ./db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      timeout: 3s
      retries: 10
    # KHÔNG mở cổng ra ngoài ở môi trường thật; ở prototype thì tiện cho việc gỡ lỗi
    ports:
      - "${DB_PORT:-5432}:5432"

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgres://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
      JWT_SECRET:   ${JWT_SECRET}
      PORT:         8000
      LOG_LEVEL:    ${LOG_LEVEL:-info}
    depends_on:
      db:
        condition: service_healthy      # CHỜ db sẵn sàng, không chỉ chờ db khởi động
    ports:
      - "${API_PORT:-8000}:8000"

  web:
    build:
      context: ./frontend
      args:
        VITE_API_BASE_URL: ${PUBLIC_API_URL:-http://localhost:8000}
    depends_on:
      api:
        condition: service_healthy      # dùng HEALTHCHECK trong backend/Dockerfile
    ports:
      - "${WEB_PORT:-5173}:80"

volumes:
  db_data:
```

.env.example — commit tệp này, KHÔNG commit .env

```
# Cơ sở dữ liệu
DB_NAME=mekong_crm
DB_USER=app_user
DB_PASSWORD=doi_mat_khau_nay
DB_PORT=5432

# API
API_PORT=8000
JWT_SECRET=chuoi_bi_mat_it_nhat_32_ky_tu_hay_sinh_ngau_nhien
LOG_LEVEL=info

# Frontend
WEB_PORT=5173
PUBLIC_API_URL=http://localhost:8000
```

Vì sao "condition: service_healthy" quan trọng hơn "depends_on" đơn thuần

depends_on đơn thuần chỉ bảo đảm container db được KHỞI ĐỘNG trước container api. Nhưng PostgreSQL mất vài giây để sẵn sàng nhận kết nối.

Trên máy em, ứng dụng có thể chậm hơn nên vô tình vẫn kết nối được. Trên máy nhanh hơn, ứng dụng thử kết nối trước khi cơ sở dữ liệu sẵn sàng và thoát ngay với lỗi ECONNREFUSED.

Đây chính xác là kiểu lỗi khiến gói đóng gói "trên máy em chạy được" mà trên máy giảng viên thì không. Nó cũng là kiểu lỗi khó chẩn đoán nhất vì nó phụ thuộc vào thời điểm.

Phương án dự phòng: thêm vòng chờ kết nối trong mã khởi động ứng dụng (thử kết nối, thất bại thì chờ 2 giây rồi thử lại, tối đa 15 lần). Làm cả hai thì an toàn nhất.

Về thư mục khởi tạo: PostgreSQL chỉ chạy các tệp nằm TRỰC TIẾP trong /docker-entrypoint-initdb.d, bỏ qua thư mục con, và chỉ chạy khi volume còn trống. Sửa SQL rồi thì phải docker compose down -v. Nếu em chạy migration bằng công cụ của framework (Knex, Prisma, Sequelize), hãy cho api chạy lệnh migrate khi khởi động thay vì dùng thư mục này.

A3. Logging ra thiết bị xuất chuẩn, có mã tương quan

src/logger.js — log dạng JSON, có requestId để lần theo một yêu cầu

```
const { randomUUID } = require('crypto');

const MUC = { debug: 10, info: 20, warn: 30, error: 40 };
const MUC_TOI_THIEU = MUC[process.env.LOG_LEVEL || 'info'];

// Các trường TUYỆT ĐỐI KHÔNG được ghi vào log
const TRUONG_CHE = ['password', 'mat_khau', 'token', 'jwt', 'authorization',
                    'cvv', 'so_the', 'secret', 'so_dien_thoai', 'sodienthoai'];

function che(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Error) return { loai: obj.name, thong_diep: obj.message, stack: obj.stack };
  const out = Array.isArray(obj) ? [] : {};
  for (const [k, v] of Object.entries(obj)) {
    out[k] = TRUONG_CHE.some(t => k.toLowerCase().includes(t))
      ? '***' : che(v);
  }
  return out;
}

function ghi(muc, thongDiep, ngucanh = {}) {
  if (MUC[muc] < MUC_TOI_THIEU) return;
  // Ghi ra STDOUT dạng một dòng JSON - đọc được bằng docker compose logs
  process.stdout.write(JSON.stringify({
    thoi_diem: new Date().toISOString(),
    muc,
    thong_diep: thongDiep,
    ...che(ngucanh),
  }) + '\n');
}

const logger = {
  debug: (m, c) => ghi('debug', m, c),
  info:  (m, c) => ghi('info',  m, c),
  warn:  (m, c) => ghi('warn',  m, c),
  error: (m, c) => ghi('error', m, c),
};

// Middleware gán một requestId cho mỗi yêu cầu, để lần theo một yêu cầu
// qua nhiều dòng log khác nhau - đây là thứ cứu em rất nhiều thời gian gỡ lỗi
function ganRequestId(req, res, next) {
  req.requestId = req.headers['x-request-id'] || randomUUID();
  res.setHeader('X-Request-Id', req.requestId);
  const t0 = Date.now();
  res.on('finish', () => logger.info('yêu cầu hoàn tất', {
    requestId: req.requestId, phuong_thuc: req.method, duong_dan: req.path,
    ma_trang_thai: res.statusCode, thoi_gian_ms: Date.now() - t0,
  }));
  next();
}

module.exports = { logger, ganRequestId };
```

Ba thứ mọi dòng log phải có · một thứ tuyệt đối không được có

PHẢI CÓ 1 — thời điểm theo chuẩn ISO. Không có thời điểm thì log vô dụng khi cần đối chiếu với báo cáo của người dùng.

PHẢI CÓ 2 — mã tương quan (requestId). Một yêu cầu sinh ra năm dòng log rải rác; không có mã tương quan thì em không biết dòng nào thuộc yêu cầu nào khi có nhiều người dùng cùng lúc.

PHẢI CÓ 3 — ngữ cảnh đủ để tái lập. Mã phiếu, mã người dùng, giá trị đầu vào đã che thông tin nhạy cảm. "Lỗi khi tạo phiếu" không giúp gì; "Lỗi khi tạo phiếu, serial SN10293A, center_id 2" thì tái lập được.

TUYỆT ĐỐI KHÔNG — mật khẩu, token, số thẻ, chuỗi kết nối đầy đủ, số điện thoại khách (QT-15). Log thường được gửi tới hệ thống bên thứ ba và được nhiều người đọc. Ghi secret vào log là làm rò rỉ secret. Hàm che() ở trên là mức tối thiểu.

Bộ xử lý lỗi duy nhất của Express (middleware xuLyLoi, đăng ký cuối cùng trong app.js) đã có trong slide buổi 11. Nó dùng chính logger này: lỗi nghiệp vụ ghi INFO và trả 409/422, lỗi hệ thống ghi ERROR kèm requestId và trả 500/503 với mã tra cứu — ghi log đúng một lần, ở middleware.

A4. Phép thử tự kiểm [SE]

Chạy đủ năm bước này trước khi tới buổi 12

```
# 1. Dựng lại từ con số không
docker compose down -v && docker compose up -d --build

# 2. Chờ tới khi mọi dịch vụ ở trạng thái healthy (không phải chỉ "running")
docker compose ps          # cột STATUS phải là "healthy" với db và api

# 3. Kiểm tra API sống và đã có dữ liệu mẫu
curl -s http://localhost:8000/health          # {"status":"ok"}
curl -s http://localhost:8000/api/tickets | head -c 300
#    (API cần đăng nhập thì lấy token bằng tài khoản mẫu trong seed trước)

# 4. Mở giao diện và bấm được ít nhất một màn hình
#    http://localhost:5173

# 5. Đọc log - phải thấy log dạng JSON có requestId
docker compose logs api --tail 20

# PHÉP THỬ CUỐI: xoá hết rồi làm lại, và PHẢI ra kết quả giống hệt lần đầu
docker compose down -v
docker compose up -d --build
```

- ☐ docker compose up -d --build từ trạng thái đã xoá sạch chạy được, không cần can thiệp tay.
- ☐ Cột STATUS của db và api là "healthy", không chỉ là "running".
- ☐ API trả về dữ liệu mẫu (seed đã chạy tự động) — hệ thống không lên trống trơn.
- ☐ Không có chuỗi kết nối, mật khẩu hay khoá bí mật nào trong repo.
- ☐ Có .env.example liệt kê đủ mọi biến; .env thật nằm trong .gitignore.
- ☐ Ảnh nền Docker ghim phiên bản cụ thể, không dùng thẻ latest.
- ☐ Container chạy bằng người dùng không đặc quyền.
- ☐ Log ra thiết bị xuất chuẩn, dạng JSON, có requestId, và không chứa secret hay số điện thoại.
- ☐ grep tìm "/home/", "/Users/", "C:\" trong mã nguồn trả về kết quả trống.

- ☐ README ghi đúng hai lệnh để chạy toàn hệ thống: cp .env.example .env rồi docker compose up -d --build.

PHẦN B — Track DA: Nối pipeline → kho dữ liệu → dashboard

B1. Đóng gói ba thành phần trong một compose

docker-compose.yml cho một hệ thống dữ liệu

```
services:

  warehouse:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB:       ${DWH_NAME}
      POSTGRES_USER:     ${DWH_USER}
      POSTGRES_PASSWORD: ${DWH_PASSWORD}
    volumes:
      - dwh_data:/var/lib/postgresql/data
      - ./sql:/docker-entrypoint-initdb.d:ro     # tạo staging, dim, fact, reject
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DWH_USER} -d ${DWH_NAME}"]
      interval: 5s
      retries: 10
    ports:
      - "${DWH_PORT:-5433}:5432"

  # Dịch vụ chạy MỘT LẦN rồi thoát - dùng để nạp dữ liệu ban đầu
  pipeline:
    build: ./pipeline
    environment:
      DWH_URL:      postgres://${DWH_USER}:${DWH_PASSWORD}@warehouse:5432/${DWH_NAME}
      DATA_DIR:     /data
      LOG_LEVEL:    info
    volumes:
      - ${LOCAL_DATA_DIR:-./data}:/data:ro       # dữ liệu nguồn KHÔNG nằm trong ảnh
      - ./reports:/app/reports                   # báo cáo chất lượng ghi ra ngoài
    depends_on:
      warehouse:
        condition: service_healthy
    # Chạy một lần rồi thoát. Muốn chạy lại: docker compose run --rm pipeline
    restart: "no"

  dashboard:
    build: ./dashboard
    environment:
      DWH_URL: postgres://${DWH_USER}:${DWH_PASSWORD}@warehouse:5432/${DWH_NAME}
    depends_on:
      warehouse:
        condition: service_healthy
    ports:
      - "${DASHBOARD_PORT:-8501}:8501"

volumes:
  dwh_data:
```

Vì sao dữ liệu nguồn KHÔNG được nằm trong ảnh Docker

Dữ liệu nguồn thường vài chục tới vài trăm megabyte. Đưa vào ảnh làm ảnh phình to và mỗi lần dựng lại đều phải sao chép lại toàn bộ.

Dữ liệu thật thường có thông tin cá nhân. Ảnh Docker được chia sẻ và đẩy lên registry, nên đưa dữ liệu vào ảnh là làm rò rỉ dữ liệu.

Cách đúng: gắn thư mục dữ liệu vào container ở chế độ chỉ đọc (:ro) qua volume, và ghi đường dẫn thư mục đó vào .env.example. Người khác đặt dữ liệu của họ vào thư mục tương ứng.

Với Bài tập 2 và 3: nộp kèm một tệp dữ liệu mẫu nhỏ (vài trăm dòng) trong repo để người chấm chạy thử được, và ghi trong README rằng dữ liệu đầy đủ nằm ở thư mục do biến LOCAL_DATA_DIR trỏ tới.

B2. Điều phối ba pha bằng một lệnh, có xử lý thất bại

src/pipeline.py — điều phối và báo cáo, có mã lô và xử lý lỗi từng pha

```
import os, sys, json, logging, datetime as dt
from src.extract import read_sources
from src.transform import clean_customers, clean_orders
from src.quality import checks
from src.load import load_staging, load_warehouse
from src.config import doc_cau_hinh, doc_tham_so

log = logging.getLogger('pipeline')

def chay(cfg, tu_ngay, den_ngay):
    ma_lo = f"{dt.datetime.now():%Y%m%d_%H%M%S}"
    log.info('bắt đầu lô %s (%s -> %s)', ma_lo, tu_ngay, den_ngay)
    bao_cao = {'ma_lo': ma_lo, 'tu_ngay': str(tu_ngay), 'den_ngay': str(den_ngay),
               'cac_pha': [], 'ket_qua_kiem': []}

    try:
        # --- PHA 1: EXTRACT ---
        tho = read_sources.doc_tat_ca(cfg['data_dir'], tu_ngay, den_ngay)
        bao_cao['cac_pha'].append({'pha': 'extract', 'trang_thai': 'OK',
                                   'so_dong': {k: len(v) for k, v in tho.items()}})

        # --- PHA 2: TRANSFORM ---
        sach = {
            'customers': clean_customers.lam_sach(tho['customers_raw']),
            'orders':    clean_orders.lam_sach(tho['orders']),
        }
        bao_cao['cac_pha'].append({'pha': 'transform', 'trang_thai': 'OK',
                                   'so_dong': {k: len(v) for k, v in sach.items()}})

        # --- PHA 3: QUALITY - chạy TRƯỚC khi nạp, để không nạp dữ liệu rác ---
        # (trích 3 hàm; BT2 yêu cầu ≥ 5 hàm kiểm chất lượng - mục D2)
        kq = [checks.cl1_dien_thoai_hop_le(sach['customers']),
              checks.cl2_ngay_hop_le(sach['orders']),
              checks.cl4_khoa_khong_trung(sach['orders'])]
        bao_cao['ket_qua_kiem'] = [
            {'ma': k.ma, 'ten': k.ten, 'so_dong_loi': k.so_dong_loi,
             'ti_le_loi': round(k.ti_le_loi, 4), 'dat': k.dat} for k in kq]

        khong_dat = [k for k in kq if not k.dat]
        if khong_dat:
            # DỪNG LẠI. Nạp dữ liệu không đạt ngưỡng là làm hỏng kho dữ liệu.
            for k in khong_dat:
                log.error('quy tắc %s KHÔNG ĐẠT: %.2f%% > ngưỡng %.2f%%',
                          k.ma, k.ti_le_loi * 100, k.nguong_cho_phep * 100)
            bao_cao['ket_luan'] = 'DUNG_VI_CHAT_LUONG'
            return bao_cao, 2

        # --- PHA 4: LOAD ---
        load_staging.nap(sach, ma_lo)
        load_warehouse.nap_dim_va_fact(tu_ngay, den_ngay)
        bao_cao['cac_pha'].append({'pha': 'load', 'trang_thai': 'OK'})
        bao_cao['ket_luan'] = 'THANH_CONG'
        return bao_cao, 0

    except Exception as e:
        log.exception('lô %s thất bại ở pha đang chạy', ma_lo)
        bao_cao['ket_luan'] = 'LOI'
        bao_cao['loi'] = f'{type(e).__name__}: {e}'
        return bao_cao, 1

    finally:
        # BÁO CÁO LUÔN ĐƯỢC GHI, kể cả khi thất bại - đây là điểm quan trọng
        os.makedirs('reports', exist_ok=True)
        with open(f'reports/run_{ma_lo}.json', 'w', encoding='utf-8') as f:
            json.dump(bao_cao, f, ensure_ascii=False, indent=2)

if __name__ == '__main__':
    _, ma_thoat = chay(doc_cau_hinh(), *doc_tham_so(sys.argv[1:]))
    sys.exit(ma_thoat)        # 0 thành công · 1 lỗi kỹ thuật · 2 dừng vì chất lượng
```

Bốn quyết định thiết kế trong đoạn mã trên — hãy giải thích được từng cái

1. Kiểm chất lượng chạy TRƯỚC khi nạp. Nạp rồi mới kiểm thì kho dữ liệu đã bị làm bẩn và các báo cáo đã sai. Thứ tự extract → transform → quality → load không thể đảo.

2. Không đạt ngưỡng thì DỪNG, không nạp một phần. Nạp một phần là trạng thái tệ nhất: kho dữ liệu vừa thiếu vừa không ai biết là thiếu. Dừng lại và báo lỗi rõ ràng thì người vận hành xử lý được.

3. Báo cáo luôn được ghi, kể cả khi thất bại. Khối finally bảo đảm điều này. Lần chạy thất bại là lần chạy cần báo cáo nhất — không có báo cáo thì không ai biết vì sao nó thất bại.

4. Mã thoát khác nhau cho các loại kết thúc khác nhau. 0 thành công · 1 lỗi kỹ thuật · 2 dừng vì chất lượng. Nhờ vậy công cụ hẹn lịch phân biệt được và gửi cảnh báo khác nhau. Đây cũng là "xử lý lỗi xuyên lớp" của pipeline: các hàm kiểm chỉ TRẢ kết quả, pipeline.py là nơi duy nhất quyết định dừng và ghi log.

B3. Chạy theo lịch — ba mức, chọn mức phù hợp phạm vi học phần

Mức	Cách làm	Phù hợp với	Thuộc phạm vi học phần nào
Mức 1 — chạy tay có tham số	python -m src.pipeline --tu-ngay ... --den-ngay ...	Prototype cá nhân, chạy lại bù khi cần	ĐỦ để đạt Bài tập 2 và Bài tập 3
Mức 2 — hẹn lịch bằng công cụ hệ điều hành	crontab trên Linux hoặc Task Scheduler trên Windows, gọi lệnh của mức 1	Chạy định kỳ đơn giản, một pipeline, không phụ thuộc lẫn nhau	Được khuyến khích; ghi cấu hình vào Deployment Guide là đủ, không bắt buộc cài đặt thật
Mức 3 — hệ thống điều phối	Airflow, Dagster, Prefect: đồ thị phụ thuộc, chạy lại từng tác vụ, giao diện theo dõi	Nhiều pipeline phụ thuộc nhau, nhiều người vận hành	THUỘC CHUYÊN ĐỀ TỐT NGHIỆP 2 — không làm ở học phần này

Cấu hình hẹn lịch mức 2 — ghi vào Deployment Guide

```
# ---------- Linux / macOS: crontab -e ----------
# Chạy hằng ngày 02:00, nạp dữ liệu của NGÀY HÔM QUA.
# Crontab KHÔNG hỗ trợ xuống dòng bằng "\" - mỗi lịch phải nằm trên MỘT dòng.
0 2 * * * cd /opt/du-an && docker compose run --rm pipeline python -m src.pipeline >> /opt/du-an/logs/cron.log 2>&1

# ---------- Windows: Task Scheduler ----------
# Tạo Basic Task -> Daily 02:00
#   Program:   docker
#   Arguments: compose -f C:\du-an\docker-compose.yml run --rm pipeline
#   Start in:  C:\du-an

# ---------- Chạy bù khi một ngày bị thất bại ----------
docker compose run --rm pipeline \
  python -m src.pipeline --tu-ngay 2026-10-20 --den-ngay 2026-10-20
# Chạy bù được là nhờ pipeline CHẠY LẠI ĐƯỢC - nếu chưa chạy lại được
# thì chạy bù sẽ nhân đôi dữ liệu. Đây là lý do mục kiểm chứng C5 quan trọng.
```

B4. Phép thử tự kiểm [DA]

Chạy đủ sáu bước này trước khi tới buổi 12

```
# 1. Dựng lại từ con số không
docker compose down -v && docker compose up -d warehouse
docker compose ps                      # warehouse phải là "healthy"

# 2. Chạy pipeline lần 1
docker compose run --rm pipeline python -m src.pipeline
cat reports/run_*.json | tail -40      # đọc báo cáo: các pha OK, kết luận THANH_CONG

# 3. Đếm số dòng ở bảng fact - GHI LẠI số này
#    (sh -c '...' để biến $POSTGRES_USER được đọc BÊN TRONG container)
docker compose exec warehouse sh -c \
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT COUNT(*) FROM fact_ticket;"'

# 4. Chạy pipeline lần 2 với ĐÚNG đầu vào đó
docker compose run --rm pipeline python -m src.pipeline
docker compose exec warehouse sh -c \
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT COUNT(*) FROM fact_ticket;"'
#    → PHẢI ra đúng số ở bước 3

# 5. Mở dashboard và kiểm cả ba trạng thái
docker compose up -d dashboard         # http://localhost:8501
#    - chọn khoảng ngày có dữ liệu  -> thấy biểu đồ
#    - chọn khoảng ngày không có    -> thấy thông báo rỗng
#    - docker compose stop warehouse -> tải lại trang -> thấy thông báo lỗi

# 6. Kiểm bảng reject có dòng trong đó (bộ dữ liệu chắc chắn có dòng lỗi)
docker compose exec warehouse sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT ma_quy_tac, COUNT(*) FROM reject_rows GROUP BY 1 ORDER BY 2 DESC;"'
```

- ☐ docker compose up từ trạng thái đã xoá sạch dựng được kho dữ liệu với đủ bảng.
- ☐ Pipeline chạy được bằng một lệnh và sinh ra tệp báo cáo trong reports/.
- ☐ Báo cáo được ghi cả khi pipeline thất bại (đã thử bằng cách cố tình đổi sai đường dẫn dữ liệu).
- ☐ Chạy lần thứ hai cho đúng số dòng như lần thứ nhất ở mọi bảng đích.
- ☐ Kiểm chất lượng chạy TRƯỚC khi nạp, và không đạt ngưỡng thì pipeline dừng chứ không nạp một phần.
- ☐ Có ≥ 5 hàm kiểm chất lượng; bảng reject có dòng trong đó, và lý do viết cho người đọc hiểu được.
- ☐ Dữ liệu nguồn gắn vào container qua volume chỉ đọc, không nằm trong ảnh Docker.
- ☐ Dashboard xử lý đúng cả ba trạng thái, thông báo lỗi không in ra chuỗi kết nối.
- ☐ Deployment Guide có mục nêu cấu hình hẹn lịch (mỗi lịch một dòng) và cách chạy bù một ngày thất bại.
- ☐ Không có chuỗi kết nối hay mật khẩu nào trong repo; có .env.example.

PHẦN C — Track AI: Đóng gói mô hình thành dịch vụ dự đoán

C1. Dịch vụ dự đoán bằng FastAPI — mã mẫu hoàn chỉnh

Mô hình đã huấn luyện chỉ là một tệp. Để nó thành một phần của hệ thống, em bọc nó trong một dịch vụ có giao diện rõ ràng. FastAPI phù hợp vì nó tự kiểm tra dữ liệu vào (pydantic, trả 422 khi sai) và tự sinh tài liệu API tại /docs.

src/serve/app.py — dịch vụ dự đoán, nạp mô hình MỘT LẦN khi khởi động

```
import os, time, logging, joblib
import numpy as np
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from src.features.text import lam_sach

log = logging.getLogger('serve')
MODEL_PATH = os.environ.get('MODEL_PATH', 'artifacts/model_v1.joblib')
NGUONG_TIN_CAY = float(os.environ.get('NGUONG_TIN_CAY', '0.45'))

trang_thai = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Nạp mô hình MỘT LẦN khi khởi động, không nạp mỗi lần có yêu cầu
    goi = joblib.load(MODEL_PATH)
    trang_thai['vectorizer']  = goi['vectorizer']
    trang_thai['model']       = goi['model']
    trang_thai['nhan_labels'] = goi['nhan_labels']
    trang_thai['phien_ban']   = goi.get('phien_ban_du_lieu', 'khong_ro')
    log.info('đã nạp mô hình từ %s', MODEL_PATH)
    yield
    trang_thai.clear()

app = FastAPI(title='Phân loại sự cố – Mekong Mobile', version='1.0',
              lifespan=lifespan)

class YeuCau(BaseModel):
    mo_ta: str = Field(min_length=5, max_length=2000,
                       description='Mô tả sự cố do nhân viên tiếp nhận nhập')

class NhanDuDoan(BaseModel):
    nhan: str
    do_tin_cay: float

class PhanHoi(BaseModel):
    nhan_goi_y: str | None       # None khi không đủ tin cậy
    do_tin_cay: float
    top_3: list[NhanDuDoan]
    can_nguoi_xem_lai: bool
    phien_ban_mo_hinh: str

@app.get('/health')
def health():
    """Endpoint để docker-compose và người vận hành biết dịch vụ có sống."""
    if 'model' not in trang_thai:
        raise HTTPException(503, 'Mô hình chưa nạp xong')
    return {'status': 'ok', 'phien_ban_mo_hinh': trang_thai['phien_ban']}

@app.post('/du-doan', response_model=PhanHoi)
def du_doan(req: YeuCau):
    t0 = time.perf_counter()
    x = trang_thai['vectorizer'].transform([lam_sach(req.mo_ta)])

    if hasattr(trang_thai['model'], 'predict_proba'):
        xac_suat = trang_thai['model'].predict_proba(x)[0]
    else:
        # LinearSVC không có predict_proba - dùng điểm quyết định chuẩn hoá
        d = trang_thai['model'].decision_function(x)[0]
        e = np.exp(d - d.max()); xac_suat = e / e.sum()

    thu_tu = xac_suat.argsort()[::-1]
    top3 = [NhanDuDoan(nhan=trang_thai['nhan_labels'][i],
                       do_tin_cay=round(float(xac_suat[i]), 4))
            for i in thu_tu[:3]]

    du_tin_cay = top3[0].do_tin_cay >= NGUONG_TIN_CAY
    log.info('dự đoán xong: tin cậy=%.2f, %.1f ms', top3[0].do_tin_cay,
             (time.perf_counter() - t0) * 1000)      # KHÔNG ghi nguyên văn mô tả
    return PhanHoi(
        nhan_goi_y=top3[0].nhan if du_tin_cay else None,
```

```
        do_tin_cay=top3[0].do_tin_cay,
        top_3=top3,
        can_nguoi_xem_lai=not du_tin_cay,
        phien_ban_mo_hinh=trang_thai['phien_ban'],
    )
```

Năm quyết định thiết kế trong dịch vụ trên — mỗi cái là một câu trả lời cho câu hỏi chuẩn

1. Nạp mô hình một lần khi khởi động, không nạp mỗi lần có yêu cầu. Nạp mô hình mất hàng trăm mili giây; nạp lại mỗi lần biến dịch vụ nhanh thành dịch vụ chậm gấp trăm lần.

2. Trả về top 3 chứ không chỉ một nhãn. Nhân viên tiếp nhận cần thấy các lựa chọn gần nhất để tự quyết. Một nhãn duy nhất buộc họ tin hoặc bỏ, không có ở giữa.

3. Có ngưỡng tin cậy và cờ "cần người xem lại". Mô hình không chắc thì nói ra là không chắc. Đây là điểm giao giữa kỹ thuật và đạo đức: hệ thống không được giả vờ chắc chắn.

4. Trả về phiên bản mô hình trong mọi phản hồi. Ba tháng sau, khi có người báo "hệ thống gợi ý sai", em cần biết lúc đó đang chạy mô hình nào. Đây là nguyên tắc TRUY VẾT ĐƯỢC.

5. Có endpoint /health riêng. Để docker-compose biết khi nào dịch vụ sẵn sàng, và để người vận hành kiểm tra nhanh mà không phải gửi một yêu cầu dự đoán thật.

C2. Dockerfile cho dịch vụ mô hình — mô hình đi vào ảnh hay không?

Phương án	Cách làm	Ưu điểm	Nhược điểm
Mô hình nằm TRONG ảnh	COPY artifacts/model_v1.joblib vào ảnh khi dựng	Chạy được ngay, không phụ thuộc gì bên ngoài; phù hợp nhất với Bài tập 3 vì gói đóng gói tự đủ	Ảnh lớn; đổi mô hình phải dựng lại ảnh; phải tạo mô hình trước khi dựng
Mô hình gắn qua volume	Gắn thư mục artifacts vào container khi chạy	Đổi mô hình không cần dựng lại ảnh	Gói không tự đủ; người khác phải có tệp mô hình mới chạy được — không phù hợp phép thử máy trắng
Mô hình tải từ nơi lưu trữ khi khởi động	Dịch vụ tải mô hình từ một địa chỉ khi khởi động	Quản lý được nhiều phiên bản mô hình	Cần nơi lưu trữ và mạng; thuộc phạm vi Chuyên đề tốt nghiệp 2

serve/Dockerfile — phương án 1, phù hợp Bài tập 3

```
FROM python:3.12-slim AS runtime
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

# Cài phụ thuộc trước, tận dụng cache
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Chạy bằng người dùng không đặc quyền
RUN useradd -m -u 10001 app

COPY src ./src
COPY artifacts/model_v1.joblib ./artifacts/model_v1.joblib

USER app
ENV MODEL_PATH=artifacts/model_v1.joblib NGUONG_TIN_CAY=0.45
EXPOSE 8080

HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=6 \
  CMD python -c "import urllib.request,sys; \
    sys.exit(0 if urllib.request.urlopen('http://localhost:8080/health').status==200 else 1)"

CMD ["uvicorn", "src.serve.app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Ba điều phải làm trước khi dựng ảnh có mô hình bên trong

Chạy lại script huấn luyện để sinh mô hình mới nhất — ảnh sẽ chứa đúng tệp có ở thời điểm dựng.

Kiểm phiên bản thư viện: mô hình được lưu bởi scikit-learn 1.5.2 mà ảnh cài 1.6 có thể nạp không được hoặc cho kết quả khác. requirements.txt phải ghim phiên bản CHÍNH XÁC bằng dấu ==.

Ghi vào README lệnh sinh lại mô hình trước khi dựng ảnh, ví dụ: python -m src.models.train --config configs/train.yaml && docker compose build. Người chấm phải làm được cả hai bước.

C3. Ghép dịch vụ mô hình vào hệ thống

docker-compose.yml — ứng dụng gọi dịch vụ mô hình (db giống phần A2)

```
services:

  model:
    build: ./serve
    environment:
      NGUONG_TIN_CAY: ${NGUONG_TIN_CAY:-0.45}
      LOG_LEVEL:      info
    ports:
      - "${MODEL_PORT:-8080}:8080"

  api:
    build: ./backend
    environment:
      # Gọi bằng TÊN DỊCH VỤ trong mạng của compose, không dùng localhost
      MODEL_URL:        http://model:8080
      MODEL_TIMEOUT_MS: "1500"
      DATABASE_URL:     postgres://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    depends_on:
      db:
        condition: service_healthy    # CSDL là thành phần BẮT BUỘC → phải chờ healthy
      model:
        condition: service_started    # mô hình là PHỤ TRỢ → không chặn api khởi động
    ports:
      - "${API_PORT:-8000}:8000"
```

Gọi dịch vụ mô hình có xử lý thất bại — điểm ăn điểm ở Bài tập 2

```
async function goiYNhomLoi(moTa, requestId) {
  const ctrl = new AbortController();
  const hetHan = setTimeout(() => ctrl.abort(),
                            Number(process.env.MODEL_TIMEOUT_MS || 1500));
  try {
    const res = await fetch(`${process.env.MODEL_URL}/du-doan`, {
      method: 'POST', signal: ctrl.signal,
      headers: { 'Content-Type': 'application/json', 'X-Request-Id': requestId },
      body: JSON.stringify({ mo_ta: moTa }),
    });
    if (!res.ok) throw new Error(`model trả về ${res.status}`);
    const kq = await res.json();
    return { nguon: 'mo_hinh', ...kq };
  } catch (e) {
    // SUY GIẢM CÓ KIỂM SOÁT: mô hình chết thì hệ thống VẪN CHẠY,
    // chỉ mất phần gợi ý. Nhân viên tự chọn nhóm lỗi như trước đây.
    logger.warn('không gọi được dịch vụ mô hình, chuyển sang chọn tay',
                { requestId, ly_do: e.message });
    return { nguon: 'chon_tay', nhan_goi_y: null,
             can_nguoi_xem_lai: true };
  } finally {
    clearTimeout(hetHan);
  }
}
```

Vì sao đoạn xử lý thất bại trên là phần quan trọng nhất của mục C3

Dịch vụ mô hình là một thành phần có thể chết: hết bộ nhớ, mô hình nạp lỗi, mạng nội bộ chậm. Đây là điều bình thường, không phải trường hợp hiếm.

Nếu không xử lý: mô hình chết làm cả chức năng tiếp nhận phiếu chết theo. Một tính năng phụ trợ đánh sập nghiệp vụ chính — đây là lỗi thiết kế nghiêm trọng.

Nếu xử lý đúng (suy giảm có kiểm soát): mô hình chết thì nhân viên chọn nhóm lỗi bằng tay như trước khi có AI. Hệ thống vẫn chạy, chỉ mất phần tiện lợi.

Bắt buộc phải có thời gian chờ tối đa. Không có timeout thì một dịch vụ mô hình treo sẽ làm mọi yêu cầu tiếp nhận phiếu treo theo, và hệ thống sập vì cạn kết nối.

Nhất quán với compose: api chỉ chờ model ở mức service_started, không chờ healthy. Nếu bắt api chờ model healthy thì khi mô hình hỏng, api không khởi động được — trái với chính nguyên tắc suy giảm có kiểm soát.

Đây là dạng câu hỏi của câu 3 đề thi cuối kỳ: "hệ thống phát sinh lỗi sau khi triển khai, phân tích nguyên nhân và đề xuất giải pháp".

C4. Phép thử tự kiểm [AI]

Chạy đủ sáu bước này trước khi tới buổi 12

```
# 1. Sinh lại mô hình rồi dựng ảnh
python -m src.models.train --config configs/train.yaml
docker compose down -v && docker compose up -d --build

# 2. Chờ dịch vụ mô hình sẵn sàng
docker compose ps                           # model phải là "healthy"
curl -s http://localhost:8080/health        # {"status":"ok","phien_ban_mo_hinh":...}

# 3. Gọi dự đoán với một mô tả THẬT từ dữ liệu
curl -s -X POST http://localhost:8080/du-doan \
  -H 'Content-Type: application/json' \
  -d '{"mo_ta":"Máy sạc không vào điện, cắm sạc không lên"}' | python -m json.tool

# 4. Gọi với dữ liệu SAI - phải trả về 422 kèm thông báo rõ ràng
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/du-doan \
  -H 'Content-Type: application/json' -d '{"mo_ta":"abc"}'      # kỳ vọng 422

# 5. THỬ SUY GIẢM CÓ KIỂM SOÁT - bước quan trọng nhất
docker compose stop model
#    Bây giờ gọi chức năng tiếp nhận phiếu qua API chính:
curl -s -X POST http://localhost:8000/api/tickets -H 'Content-Type: application/json' \
  -d '{"serialNo":"SN10293A","customerId":1024,"mucDo":"CAO","moTaLoi":"Máy sạc không vào điện"}'
#    PHẢI vẫn tạo được phiếu, chỉ không có nhãn gợi ý.
#    Nếu trả về 500 -> chưa xử lý thất bại, phải sửa trước khi nộp.
docker compose start model

# 6. Đọc log của dịch vụ mô hình
docker compose logs model --tail 20
```

- ☐ Mô hình được nạp một lần khi khởi động, không nạp lại mỗi lần có yêu cầu.
- ☐ Có endpoint /health và nó trả về phiên bản mô hình.
- ☐ Phản hồi dự đoán có top 3 nhãn, độ tin cậy, và cờ cần người xem lại.
- ☐ Mọi phản hồi đều trả về phiên bản mô hình.
- ☐ Ngưỡng tin cậy lấy từ biến môi trường, không ghi cứng.
- ☐ Dữ liệu vào không hợp lệ trả về 422 kèm thông báo nói rõ trường nào sai.
- ☐ Đã thử tắt dịch vụ mô hình và nghiệp vụ chính VẪN chạy được (suy giảm có kiểm soát).
- ☐ Lệnh gọi dịch vụ mô hình có thời gian chờ tối đa.
- ☐ requirements.txt ghim phiên bản chính xác; mô hình nạp được trong ảnh Docker.
- ☐ README ghi rõ hai bước: sinh lại mô hình rồi dựng ảnh.

PHẦN D — Dùng chung cho cả ba track: Bug Log, logging và xử lý lỗi

Phần này hiện thực ba nội dung lý thuyết của buổi 11 ở mức tệp tin cụ thể. Track SE đã có logger và middleware lỗi bằng Node.js (phần A3 và slide buổi 11); track DA và AI dùng mã Python dưới đây.

D1. Bug Log — ghi ngay khi gỡ xong, đừng chờ tới Bài tập 3

Mỗi lần em chạy đủ bốn bước tái lập → khoanh vùng → sửa nguyên nhân → xác nhận, em có sẵn một mục Bug Log. Bài tập 3 yêu cầu ít nhất ba mục, trong đó ít nhất một mục Critical hoặc Major, mỗi mục đủ bảy dòng và có link commit mở được.

docs/bug-log.md — mẫu một mục, đủ bảy dòng BT3 yêu cầu

```
## BUG-004 · Major — Tạo phiếu thất bại với khách thiếu họ tên

| Dòng                  | Nội dung |
|-----------------------|----------|
| 1. Mã và mức độ       | BUG-004 · Major (một nhóm khách không tạo được phiếu) |
| 2. Phát hiện bởi      | TC-07 "tạo phiếu cho khách nhập từ dữ liệu cũ" |
| 3. Triệu chứng        | POST /api/tickets trả 500; giao diện hiện "Có lỗi xảy ra" |
| 4. Cách tái lập       | curl -X POST localhost:8000/api/tickets -d @tests/fixtures/bug004.json |
| 5. Nguyên nhân        | toTicketView() gọi trim() trên full_name = null; customers_raw.csv |
|                       | có 37 bản ghi thiếu tên và khi nạp không có quy tắc kiểm |
| 6. Cách sửa           | Hàm chuyển đổi dùng "(không rõ)" khi nguồn rỗng (đúng hợp đồng); |
|                       | thêm quy tắc CL6 "thiếu họ tên" → dòng thiếu tên vào bảng reject |
| 7. Commit và xác nhận | https://github.com/<tk>/<repo>/commit/a1b2c3d |
|                       | test "tạo phiếu được khi khách thiếu họ tên" FAIL trước, PASS sau |
```

Ba lỗi khiến một mục Bug Log không được tính

Dòng 5 ghi triệu chứng thay vì nguyên nhân. "API trả về 500" là triệu chứng. Hỏi "vì sao" liên tiếp cho tới khi gặp chỗ vi phạm hợp đồng đầu tiên — đó mới là nguyên nhân.

Link commit chết hoặc trỏ vào commit không liên quan. Giảng viên sẽ bấm vào từng link. Commit sửa lỗi nên nhỏ và có thông điệp rõ: fix(ticket): xử lý khách thiếu họ tên khi tạo phiếu.

Không có test xác nhận. Bước 4 sinh ra một test hồi quy; test đó phải FAIL trước khi sửa và PASS sau khi sửa. Mô tả một bug em không thực sự gỡ là vấn đề liêm chính.

D2. Logging cho Python — cấu hình một lần, ghi JSON ra stdout

src/log_config.py — [DA/AI] cấu hình logging MỘT lần, ghi JSON ra stdout

```
import json, logging, os, sys

class DinhDangJson(logging.Formatter):
    """Mỗi dòng log là một đối tượng JSON: thời điểm, mức, nguồn, thông điệp, ngữ cảnh."""
    TRUONG_NGU_CANH = ('ma_lo', 'request_id', 'so_dong', 'ma_quy_tac')

    def format(self, r):
        d = {'thoi_diem': self.formatTime(r, '%Y-%m-%dT%H:%M:%S%z'),
             'muc': r.levelname, 'nguon': r.name, 'thong_diep': r.getMessage()}
        for k in self.TRUONG_NGU_CANH:              # chỉ lấy ngữ cảnh đã khai báo
            if hasattr(r, k): d[k] = getattr(r, k)
        if r.exc_info: d['loi'] = self.formatException(r.exc_info)
        return json.dumps(d, ensure_ascii=False)

def cau_hinh_log():
    h = logging.StreamHandler(sys.stdout)           # stdout, KHÔNG ghi tệp trong container
    h.setFormatter(DinhDangJson())
    logging.basicConfig(level=os.getenv('LOG_LEVEL', 'INFO'), handlers=[h], force=True)

# Dùng ở điểm vào (src/pipeline.py hoặc src/serve/app.py):
#   cau_hinh_log()
#   log = logging.getLogger('pipeline.transform')
#   log.warning('bỏ qua dòng không quy đổi được ngày',
#               extra={'ma_lo': ma_lo, 'so_dong': 14})
```

Ba quy tắc khi dùng logging trong Python

Gọi cau_hinh_log() đúng một lần ở điểm vào chương trình. Các module chỉ gọi logging.getLogger(__name__) — không gọi basicConfig ở từng tệp.

Không dùng print trong mã nộp bài. print không có mức, không có thời điểm, không tắt được. Dòng in để gỡ lỗi chuyển thành log.debug(...) rồi đặt LOG_LEVEL=INFO.

log.exception chỉ ở nơi QUYẾT ĐỊNH xử lý lỗi (pipeline.py, bộ xử lý lỗi của API). Không vừa log vừa ném lại ở mọi hàm — một sự cố sẽ thành ba dòng ERROR.

D3. Xử lý lỗi xuyên lớp cho API Python — một bộ xử lý duy nhất

Cùng nguyên tắc với middleware Express trong slide buổi 11: lớp nghiệp vụ ném lỗi có mã; một bộ xử lý duy nhất dịch lỗi thành mã HTTP theo định dạng { error, message } của buổi 9 và ghi log đúng một lần. Gọi dang_ky_xu_ly_loi(app) ngay sau khi tạo app trong src/serve/app.py.

src/serve/errors.py — [AI, và mọi API Python] lỗi nghiệp vụ và bộ xử lý lỗi duy nhất

```
import logging, uuid
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

log = logging.getLogger('serve.errors')

class LoiNghiepVu(Exception):
    """Vi phạm quy tắc nghiệp vụ: trả 4xx, KHÔNG phải sự cố hệ thống."""
    def __init__(self, ma: str, thong_diep: str, http_status: int = 409):
        super().__init__(thong_diep)
        self.ma, self.thong_diep, self.http_status = ma, thong_diep, http_status

def dang_ky_xu_ly_loi(app: FastAPI):
    @app.exception_handler(LoiNghiepVu)
    async def _nghiep_vu(req: Request, e: LoiNghiepVu):
        log.info('vi phạm quy tắc %s', e.ma)                    # INFO, không phải ERROR
        return JSONResponse(status_code=e.http_status,
                            content={'error': e.ma, 'message': e.thong_diep})

    @app.exception_handler(Exception)
    async def _he_thong(req: Request, e: Exception):
        ma_tra_cuu = req.headers.get('x-request-id') or uuid.uuid4().hex[:8]
        log.exception('lỗi hệ thống', extra={'request_id': ma_tra_cuu})   # log MỘT lần
        return JSONResponse(status_code=500, content={
            'error': 'LOI_HE_THONG',
            'message': f'Hệ thống đang gặp sự cố. Mã tra cứu: {ma_tra_cuu}'})
        # KHÔNG trả str(e) hay traceback cho người gọi
```

Lớp	Trách nhiệm khi có lỗi	Không được làm
Dữ liệu (repository, extract, nạp mô hình)	Bọc lỗi thư viện thành lỗi của ứng dụng, giữ nguyên nguyên nhân (cause / raise ... from e)	Trả mã HTTP; ghi log; nuốt lỗi rồi trả None
Nghiệp vụ (service, transform, quality)	Kiểm quy tắc QT-xx; vi phạm thì ném LoiNghiepVu có MÃ; không bắt lỗi hạ tầng mình không xử lý được	Biết về HTTP (req, res, status); bắt Exception chung chung
Ranh giới (middleware, exception_handler, pipeline.py)	Dịch lỗi → mã HTTP hoặc mã thoát; ghi log MỘT lần kèm requestId / mã lô	Gửi stack trace, câu SQL, chuỗi kết nối cho người dùng
Giao diện / dashboard / bên gọi	Rẽ nhánh theo error; hiện message: chuyện gì · vì sao · làm gì tiếp; có mã tra cứu với lỗi 5xx	Hiện vệt lỗi kỹ thuật; màn hình trắng

D4. Checklist chung trước workshop buổi 12

- ☐ Tệp khoá phiên bản đã commit (package-lock.json, hoặc requirements.txt ghi == tạo từ venv sạch); README ghi phiên bản runtime.
- ☐ docs/bug-log.md có ít nhất một mục đủ bảy dòng cho một lỗi em đã thực sự gỡ.
- ☐ grep -rnE "catch *\([^)]*\) *\{ *\}|except *: *pass" src/ trả về kết quả trống — không nuốt lỗi.
- ☐ Không còn print/console.log gỡ lỗi trong src/; log có mức, có thời điểm, có ngữ cảnh.
- ☐ Log không chứa mật khẩu, token, chuỗi kết nối hay số điện thoại khách.
- ☐ Lỗi người dùng trả 4xx kèm thông báo rõ; lỗi hệ thống trả 5xx với thông điệp chung và mã tra cứu.
- ☐ Đã chạy phép thử tự kiểm của track mình từ thư mục trống và ghi lại bước thất bại đầu tiên (nếu có).

So sánh nhanh ba track — bảng đối chiếu để tự kiểm

Tiêu chí	Track SE	Track DA	Track AI
Các dịch vụ trong compose	db · api · web	warehouse · pipeline · dashboard	db · model · api (· web)
Dịch vụ nào chạy một lần rồi thoát	Không có	pipeline	Không có (nhưng bước huấn luyện chạy ngoài compose)
Thứ tự khởi động quan trọng ở đâu	api chờ db healthy	pipeline và dashboard chờ warehouse healthy	api chờ db healthy; chỉ chờ model started
Dữ liệu lớn xử lý thế nào	Seed nhỏ trong repo	Dữ liệu nguồn gắn volume chỉ đọc, mẫu nhỏ trong repo	Mô hình copy vào ảnh, dữ liệu huấn luyện không vào ảnh
Log ở đâu, ghi lỗi ở đâu	logger JSON + middleware lỗi duy nhất	log JSON + pipeline.py quyết định dừng, mã thoát 0/1/2	log JSON + exception_handler duy nhất
Suy giảm có kiểm soát ở đâu	Frontend mất API → hiện trạng thái lỗi có nút thử lại	Dashboard mất kho dữ liệu → hiện thông báo lỗi rõ ràng	API mất dịch vụ mô hình → nhân viên chọn tay, nghiệp vụ vẫn chạy
Lỗi nghiêm trọng nhất của track	depends_on không kèm điều kiện healthy	Nạp dữ liệu trước khi kiểm chất lượng	Dịch vụ mô hình chết làm sập nghiệp vụ chính
Phép thử cuối cùng giống nhau cho cả ba	down -v rồi up --build từ thư mục trống	down -v rồi up --build từ thư mục trống	down -v rồi up --build từ thư mục trống

Ba việc làm ngay trong tuần này, trước workshop buổi 12 (nộp Bài tập 2)

1. Chạy phép thử tự kiểm của track mình từ đầu tới cuối, ghi ra giấy bước nào thất bại. Bước thất bại đầu tiên là việc số một của em ở buổi 12.

2. Thử SUY GIẢM CÓ KIỂM SOÁT. Tắt một dịch vụ phụ trợ và xem hệ thống còn chạy không. Nếu nó sập, sửa ngay — đây là bước 5 trong quy trình chấm Bài tập 2 và là dạng câu hỏi của câu 3 đề thi.

3. Rà lại secret. Chạy git log --all --oneline -- .env — nếu có kết quả thì .env đã từng bị commit: đổi mọi mật khẩu và khoá liên quan ngay, và ghi việc đó vào tài liệu. Xoá ở commit sau là không đủ vì secret vẫn còn trong lịch sử git.

VAN LANG UNIVERSITY · FACULTY OF INFORMATION TECHNOLOGY

SPECIALIZED GRADUATION TOPIC 1 · SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Self-study guide, Session 11 — Packaging and configuration by stack

Docker for a multi-tier web application [SE] · Connecting pipeline → warehouse → dashboard and running on a schedule [DA] · Packaging a model as a REST API with FastAPI [AI] · Bug Log, logging and cross-layer error handling [common]. Each part includes complete configuration files on the Mekong Mobile dataset and a self-check test.

The output of this guide is a precondition for submitting Assignment 2 (component 4 — an integrated system that runs) and the basis of the Assignment 3 package (runs with ≤ 2 commands). Work on it during the week of Session 11; the lecturer checks it in the Session 12 workshop.

Prepared by: **Dr. Nguyễn Trí Hải** · Department of Software Engineering

For internal circulation · Self-study series for the specialization part (30% of Session 11 theory time). Bilingual edition: this is the English version. In the sample code, identifiers, user-facing strings and log messages stay in Vietnamese because the case-study system serves Vietnamese users; code comments are translated.

How to use this guide

Taught by the lecturer in class (Session 11)	Read by students in this guide
The three sources of difference between your machine and someone else's; locking versions per stack; the four steps of systematic debugging; the Bug Log; the four log levels, the three things every log line must have and what must never be logged; cross-layer error handling; images, containers, volumes; a minimal Dockerfile; docker-compose for an application with a database.	Concrete packaging configuration for your own track's stack: several services and start-up order [SE]; wiring the three phases and scheduling [DA]; packaging a model as a prediction service [AI]. Common Part D: a Bug Log template, logging configuration for Python, a single error handler for Python APIs.

Four things that hold for all three tracks — read before your own part

1. ALL CONFIGURATION GOES INTO ENVIRONMENT VARIABLES. No connection string, password, API key or absolute path lives in the source code or in a committed configuration file. Provide a .env.example listing every variable with sample values; the real .env is in .gitignore.

2. START-UP ORDER MUST BE DEFINED. An application that starts before the database is ready is the bug that makes your package run on your machine but not on the lecturer's. Use a healthcheck with depends_on, or add a connection-retry loop at start-up.

3. LOG TO STANDARD OUTPUT, NOT TO A FILE INSIDE THE CONTAINER. A log file written inside a container is lost when the container is removed. Write to stdout so "docker compose logs" can read it — the standard convention of every container platform.

4. THE FINAL TEST IS ALWAYS THE SAME: empty folder · clone the repo · follow the README exactly · the system comes up. The lecturer runs this test in Session 12 (Assignment 2) and Session 16 (Assignment 3) — run it yourself starting this week.

Three configuration mistakes behind most "it runs on my machine" cases

Mistake	What happens on someone else's machine	How to detect it early	How to fix it
Absolute paths in the code	File-or-folder-not-found error right at start-up	grep -rnE "/home/|/Users/|C:\\" src/ across all source code	Use paths relative to the project root, or move them into environment variables
Hard-coded port that is already taken	"address already in use", or the service starts but is called on a different port	Run it while another service is using the same port	Move the port into an environment variable; explain in the README how to change it
Dependencies pre-installed on your own machine	Missing-library errors, or wrong behaviour because of a different version	Run inside a container or a brand-new virtual environment	Record exact versions in the lock file and commit it

PART A — SE track: Packaging a multi-tier web application

A1. A multi-stage Dockerfile for the back end

A multi-stage Dockerfile separates the install step from the run step. Result: a smaller runtime image with no build tools and no sensitive leftovers from the install step.

backend/Dockerfile — Node.js example

```
# ---------- STAGE 1: install dependencies ----------
FROM node:20-alpine AS deps
WORKDIR /app
# Copy ONLY the lock file first, so Docker can reuse the cached layer
# when the source code changes but the dependencies do not
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# ---------- STAGE 2: runtime image ----------
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production

# Run as a non-privileged user - a basic security requirement
RUN addgroup -S app && adduser -S app -G app

COPY --from=deps /app/node_modules ./node_modules
COPY src ./src
COPY package.json ./

USER app
EXPOSE 8000

# Healthcheck so docker-compose knows when the service is ready
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s --retries=6 \
  CMD node -e
"fetch('http://localhost:8000/health').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"

CMD ["node", "src/server.js"]
```

Four things your Dockerfile must have to pass Assignment 3

Pin a specific base-image version (node:20-alpine), never the "latest" tag — "latest" today and "latest" next month are different things, which breaks the REPRODUCIBLE principle.

Copy the lock file first and the source code afterwards — this order makes rebuilds many times faster.

Run as a non-privileged user (not root) — an item on the Session 15 security checklist.

Have a HEALTHCHECK — this lets docker-compose know when the service is really ready, rather than only that the container has started.

A2. A complete docker-compose.yml — three services and their start-up order

docker-compose.yml — copy as is, only rename the project

```
services:

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB:       ${DB_NAME}
      POSTGRES_USER:     ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
      # .sql/.sh files placed DIRECTLY in db/init run AUTOMATICALLY while the volume is empty,
      # in name order: 01_schema.sql → 02_seed.sql. Sub-folders are IGNORED.
      - ./db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      timeout: 3s
      retries: 10
    # Do NOT expose this port in a real environment; in a prototype it helps debugging
    ports:
      - "${DB_PORT:-5432}:5432"

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgres://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
      JWT_SECRET:   ${JWT_SECRET}
      PORT:         8000
      LOG_LEVEL:    ${LOG_LEVEL:-info}
    depends_on:
      db:
        condition: service_healthy      # WAIT until db is ready, not just started
    ports:
      - "${API_PORT:-8000}:8000"

  web:
    build:
      context: ./frontend
      args:
        VITE_API_BASE_URL: ${PUBLIC_API_URL:-http://localhost:8000}
    depends_on:
      api:
        condition: service_healthy      # uses the HEALTHCHECK in backend/Dockerfile
    ports:
      - "${WEB_PORT:-5173}:80"

volumes:
  db_data:
```

.env.example — commit this file, do NOT commit .env

```
# Database
DB_NAME=mekong_crm
DB_USER=app_user
DB_PASSWORD=doi_mat_khau_nay
DB_PORT=5432

# API
API_PORT=8000
JWT_SECRET=chuoi_bi_mat_it_nhat_32_ky_tu_hay_sinh_ngau_nhien
LOG_LEVEL=info

# Frontend
WEB_PORT=5173
PUBLIC_API_URL=http://localhost:8000
```

Why "condition: service_healthy" matters more than plain "depends_on"

Plain depends_on only guarantees that the db container is STARTED before the api container. But PostgreSQL needs a few seconds before it accepts connections.

On your machine the application may start more slowly and connect by luck. On a faster machine it tries to connect before the database is ready and exits immediately with ECONNREFUSED.

This is exactly the kind of bug that makes a package "run on my machine" but not on the lecturer's. It is also the hardest kind to diagnose, because it depends on timing.

Fallback: add a connection-retry loop to the application's start-up code (try to connect; on failure wait 2 seconds and retry, at most 15 times). Doing both is safest.

About the init folder: PostgreSQL only runs files placed DIRECTLY in /docker-entrypoint-initdb.d, ignores sub-folders, and only runs them while the volume is empty. After editing SQL you must run docker compose down -v. If you run migrations with your framework's tool (Knex, Prisma, Sequelize), let the api run migrate at start-up instead of using this folder.

A3. Logging to standard output, with a correlation ID

src/logger.js — JSON logs with a requestId to follow one request

```
const { randomUUID } = require('crypto');

const MUC = { debug: 10, info: 20, warn: 30, error: 40 };
const MUC_TOI_THIEU = MUC[process.env.LOG_LEVEL || 'info'];

// Fields that must NEVER be written to the log
const TRUONG_CHE = ['password', 'mat_khau', 'token', 'jwt', 'authorization',
                    'cvv', 'so_the', 'secret', 'so_dien_thoai', 'sodienthoai'];

function che(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Error) return { loai: obj.name, thong_diep: obj.message, stack: obj.stack };
  const out = Array.isArray(obj) ? [] : {};
  for (const [k, v] of Object.entries(obj)) {
    out[k] = TRUONG_CHE.some(t => k.toLowerCase().includes(t))
      ? '***' : che(v);
  }
  return out;
}

function ghi(muc, thongDiep, ngucanh = {}) {
  if (MUC[muc] < MUC_TOI_THIEU) return;
  // Write to STDOUT as one JSON line - readable with docker compose logs
  process.stdout.write(JSON.stringify({
    thoi_diem: new Date().toISOString(),
    muc,
    thong_diep: thongDiep,
    ...che(ngucanh),
  }) + '\n');
}

const logger = {
  debug: (m, c) => ghi('debug', m, c),
  info:  (m, c) => ghi('info',  m, c),
  warn:  (m, c) => ghi('warn',  m, c),
  error: (m, c) => ghi('error', m, c),
};

// Middleware that assigns a requestId to every request, so one request can be followed
// across many log lines - this will save you a lot of debugging time
function ganRequestId(req, res, next) {
  req.requestId = req.headers['x-request-id'] || randomUUID();
  res.setHeader('X-Request-Id', req.requestId);
  const t0 = Date.now();
  res.on('finish', () => logger.info('yêu cầu hoàn tất', {
    requestId: req.requestId, phuong_thuc: req.method, duong_dan: req.path,
    ma_trang_thai: res.statusCode, thoi_gian_ms: Date.now() - t0,
  }));
  next();
}

module.exports = { logger, ganRequestId };
```

Three things every log line must have · one thing it must never have

MUST HAVE 1 — an ISO timestamp. Without a timestamp the log is useless when you need to match it against a user's report.

MUST HAVE 2 — a correlation ID (requestId). One request produces five scattered log lines; without a correlation ID you cannot tell which line belongs to which request when many users are active.

MUST HAVE 3 — enough context to reproduce. Ticket code, user ID, input values with sensitive data masked. "Error creating ticket" does not help; "Error creating ticket, serial SN10293A, center_id 2" can be reproduced.

NEVER — passwords, tokens, card numbers, full connection strings, customer phone numbers (QT-15). Logs are often shipped to third-party systems and read by many people. Writing a secret into a log leaks it. The che() function above is the bare minimum.

The single Express error handler (the xuLyLoi middleware, registered last in app.js) is in the Session 11 slides. It uses this same logger: business errors are logged at INFO and return 409/422; system errors are logged at ERROR with the requestId and return 500/503 with a reference code — logged exactly once, in the middleware.

A4. Self-check test [SE]

Run all five steps before the Session 12 workshop

```
# 1. Rebuild from zero
docker compose down -v && docker compose up -d --build

# 2. Wait until every service is healthy (not merely "running")
docker compose ps          # STATUS must be "healthy" for db and api

# 3. Check that the API is alive and sample data is present
curl -s http://localhost:8000/health          # {"status":"ok"}
curl -s http://localhost:8000/api/tickets | head -c 300
#    (if the API requires login, first get a token with a seeded sample account)

# 4. Open the UI and use at least one screen
#    http://localhost:5173

# 5. Read the logs - you must see JSON lines with a requestId
docker compose logs api --tail 20

# FINAL TEST: delete everything, do it again, and you MUST get the same result
docker compose down -v
docker compose up -d --build
```

- ☐ docker compose up -d --build works from a fully cleaned state, with no manual intervention.
- ☐ The STATUS column of db and api is "healthy", not just "running".
- ☐ The API returns sample data (the seed ran automatically) — the system does not come up empty.
- ☐ No connection string, password or secret key anywhere in the repo.
- ☐ .env.example lists every variable; the real .env is in .gitignore.
- ☐ The Docker base image is pinned to a specific version, not the latest tag.
- ☐ The container runs as a non-privileged user.
- ☐ Logs go to standard output, as JSON, with a requestId, and contain no secrets or phone numbers.
- ☐ grep for "/home/", "/Users/", "C:\" in the source code returns nothing.

- ☐ The README gives exactly two commands to run the whole system: cp .env.example .env, then docker compose up -d --build.

PART B — DA track: Connecting pipeline → warehouse → dashboard

B1. Packaging three components in one compose file

docker-compose.yml for a data system

```
services:

  warehouse:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB:       ${DWH_NAME}
      POSTGRES_USER:     ${DWH_USER}
      POSTGRES_PASSWORD: ${DWH_PASSWORD}
    volumes:
      - dwh_data:/var/lib/postgresql/data
      - ./sql:/docker-entrypoint-initdb.d:ro     # creates staging, dim, fact, reject
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DWH_USER} -d ${DWH_NAME}"]
      interval: 5s
      retries: 10
    ports:
      - "${DWH_PORT:-5433}:5432"

  # A service that runs ONCE and exits - used for the initial load
  pipeline:
    build: ./pipeline
    environment:
      DWH_URL:      postgres://${DWH_USER}:${DWH_PASSWORD}@warehouse:5432/${DWH_NAME}
      DATA_DIR:     /data
      LOG_LEVEL:    info
    volumes:
      - ${LOCAL_DATA_DIR:-./data}:/data:ro       # source data is NOT inside the image
      - ./reports:/app/reports                   # quality reports are written outside
    depends_on:
      warehouse:
        condition: service_healthy
    # Runs once and exits. To run again: docker compose run --rm pipeline
    restart: "no"

  dashboard:
    build: ./dashboard
    environment:
      DWH_URL: postgres://${DWH_USER}:${DWH_PASSWORD}@warehouse:5432/${DWH_NAME}
    depends_on:
      warehouse:
        condition: service_healthy
    ports:
      - "${DASHBOARD_PORT:-8501}:8501"

volumes:
  dwh_data:
```

Why source data must NOT be inside the Docker image

Source data is often tens to hundreds of megabytes. Putting it in the image bloats the image, and every rebuild copies it all again.

Real data often contains personal information. Docker images are shared and pushed to registries, so putting data into an image leaks it.

The right way: mount the data folder into the container read-only (:ro) through a volume, and record that folder path in .env.example. Other people put their own data in the corresponding folder.

For Assignments 2 and 3: include a small sample data file (a few hundred rows) in the repo so the grader can run it, and state in the README that the full data lives in the folder pointed to by LOCAL_DATA_DIR.

B2. Orchestrating the three phases with one command, with failure handling

src/pipeline.py — orchestration and reporting, with a batch ID and per-phase error handling

```
import os, sys, json, logging, datetime as dt
from src.extract import read_sources
from src.transform import clean_customers, clean_orders
from src.quality import checks
from src.load import load_staging, load_warehouse
from src.config import doc_cau_hinh, doc_tham_so

log = logging.getLogger('pipeline')

def chay(cfg, tu_ngay, den_ngay):
    ma_lo = f"{dt.datetime.now():%Y%m%d_%H%M%S}"
    log.info('bắt đầu lô %s (%s -> %s)', ma_lo, tu_ngay, den_ngay)
    bao_cao = {'ma_lo': ma_lo, 'tu_ngay': str(tu_ngay), 'den_ngay': str(den_ngay),
               'cac_pha': [], 'ket_qua_kiem': []}

    try:
        # --- PHASE 1: EXTRACT ---
        tho = read_sources.doc_tat_ca(cfg['data_dir'], tu_ngay, den_ngay)
        bao_cao['cac_pha'].append({'pha': 'extract', 'trang_thai': 'OK',
                                   'so_dong': {k: len(v) for k, v in tho.items()}})

        # --- PHASE 2: TRANSFORM ---
        sach = {
            'customers': clean_customers.lam_sach(tho['customers_raw']),
            'orders':    clean_orders.lam_sach(tho['orders']),
        }
        bao_cao['cac_pha'].append({'pha': 'transform', 'trang_thai': 'OK',
                                   'so_dong': {k: len(v) for k, v in sach.items()}})

        # --- PHASE 3: QUALITY - runs BEFORE loading, so no junk gets loaded ---
        # (3 shown; Assignment 2 requires ≥ 5 quality checks - item D2)
        kq = [checks.cl1_dien_thoai_hop_le(sach['customers']),
              checks.cl2_ngay_hop_le(sach['orders']),
              checks.cl4_khoa_khong_trung(sach['orders'])]
        bao_cao['ket_qua_kiem'] = [
            {'ma': k.ma, 'ten': k.ten, 'so_dong_loi': k.so_dong_loi,
             'ti_le_loi': round(k.ti_le_loi, 4), 'dat': k.dat} for k in kq]

        khong_dat = [k for k in kq if not k.dat]
        if khong_dat:
            # STOP. Loading data that fails the threshold corrupts the warehouse.
            for k in khong_dat:
                log.error('quy tắc %s KHÔNG ĐẠT: %.2f%% > ngưỡng %.2f%%',
                          k.ma, k.ti_le_loi * 100, k.nguong_cho_phep * 100)
            bao_cao['ket_luan'] = 'DUNG_VI_CHAT_LUONG'
            return bao_cao, 2

        # --- PHASE 4: LOAD ---
        load_staging.nap(sach, ma_lo)
        load_warehouse.nap_dim_va_fact(tu_ngay, den_ngay)
        bao_cao['cac_pha'].append({'pha': 'load', 'trang_thai': 'OK'})
        bao_cao['ket_luan'] = 'THANH_CONG'
        return bao_cao, 0

    except Exception as e:
        log.exception('lô %s thất bại ở pha đang chạy', ma_lo)
        bao_cao['ket_luan'] = 'LOI'
        bao_cao['loi'] = f'{type(e).__name__}: {e}'
        return bao_cao, 1

    finally:
        # The REPORT IS ALWAYS WRITTEN, even on failure - this is the key point
        os.makedirs('reports', exist_ok=True)
        with open(f'reports/run_{ma_lo}.json', 'w', encoding='utf-8') as f:
            json.dump(bao_cao, f, ensure_ascii=False, indent=2)

if __name__ == '__main__':
    _, ma_thoat = chay(doc_cau_hinh(), *doc_tham_so(sys.argv[1:]))
    sys.exit(ma_thoat)        # 0 success · 1 technical error · 2 stopped for quality
```

Four design decisions in the code above — be able to explain each one

1. Quality checks run BEFORE loading. If you load first and check afterwards, the warehouse is already polluted and the reports are already wrong. The order extract → transform → quality → load cannot be swapped.

2. Below threshold, STOP — do not load partially. A partial load is the worst state: the warehouse is incomplete and nobody knows it. Stopping with a clear error lets the operator deal with it.

3. The report is always written, even on failure. The finally block guarantees this. A failed run is the run that needs a report most — without one nobody knows why it failed.

4. Different exit codes for different outcomes. 0 success · 1 technical error · 2 stopped for quality. The scheduler can then tell them apart and raise different alerts. This is also the pipeline's "cross-layer error handling": check functions only RETURN results; pipeline.py is the single place that decides to stop and writes the log.

B3. Running on a schedule — three levels; choose the one that fits this course

Level	How	Suitable for	Which course it belongs to
Level 1 — manual run with parameters	python -m src.pipeline --tu-ngay ... --den-ngay ...	A personal prototype; backfilling when needed	ENOUGH to pass Assignments 2 and 3
Level 2 — OS scheduler	crontab on Linux or Task Scheduler on Windows, calling the level-1 command	Simple periodic runs, a single pipeline, no inter-dependencies	Encouraged; documenting the configuration in the Deployment Guide is enough, no real installation required
Level 3 — orchestration system	Airflow, Dagster, Prefect: dependency graphs, per-task reruns, monitoring UI	Many inter-dependent pipelines, several operators	BELONGS TO SPECIALIZED GRADUATION TOPIC 2 — not in this course

Level-2 scheduling configuration — write it into the Deployment Guide

```
# ---------- Linux / macOS: crontab -e ----------
# Runs daily at 02:00, loading YESTERDAY's data.
# Crontab does NOT support "\" line continuation - each schedule must be ONE line.
0 2 * * * cd /opt/du-an && docker compose run --rm pipeline python -m src.pipeline >> /opt/du-an/logs/cron.log 2>&1

# ---------- Windows: Task Scheduler ----------
# Create Basic Task -> Daily 02:00
#   Program:   docker
#   Arguments: compose -f C:\du-an\docker-compose.yml run --rm pipeline
#   Start in:  C:\du-an

# ---------- Backfill a day that failed ----------
docker compose run --rm pipeline \
  python -m src.pipeline --tu-ngay 2026-10-20 --den-ngay 2026-10-20
# Backfilling works only because the pipeline is RE-RUNNABLE - if it is not,
# a backfill duplicates data. This is why check item C5 matters.
```

B4. Self-check test [DA]

Run all six steps before the Session 12 workshop

```
# 1. Rebuild from zero
docker compose down -v && docker compose up -d warehouse
docker compose ps                      # warehouse must be "healthy"

# 2. Run the pipeline the first time
docker compose run --rm pipeline python -m src.pipeline
cat reports/run_*.json | tail -40      # read the report: phases OK, conclusion THANH_CONG

# 3. Count the rows in the fact table - WRITE THIS NUMBER DOWN
#    (sh -c '...' so $POSTGRES_USER is read INSIDE the container)
docker compose exec warehouse sh -c \
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT COUNT(*) FROM fact_ticket;"'

# 4. Run the pipeline a second time with EXACTLY the same input
docker compose run --rm pipeline python -m src.pipeline
docker compose exec warehouse sh -c \
  'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT COUNT(*) FROM fact_ticket;"'
#    → MUST give exactly the number from step 3

# 5. Open the dashboard and check all three states
docker compose up -d dashboard         # http://localhost:8501
#    - pick a date range with data     -> you see the charts
#    - pick a date range without data  -> you see the empty message
#    - docker compose stop warehouse -> reload -> you see the error message

# 6. Check that the reject table has rows (the dataset certainly has bad rows)
docker compose exec warehouse sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT ma_quy_tac, COUNT(*) FROM reject_rows GROUP BY 1 ORDER BY 2 DESC;"'
```

- ☐ docker compose up from a fully cleaned state builds the warehouse with all tables.
- ☐ The pipeline runs with one command and produces a report file in reports/.
- ☐ The report is written even when the pipeline fails (tested by deliberately breaking the data path).
- ☐ A second run gives exactly the same row counts as the first in every target table.
- ☐ Quality checks run BEFORE loading, and below threshold the pipeline stops instead of loading partially.
- ☐ There are ≥ 5 quality-check functions; the reject table has rows, with reasons a reader can understand.
- ☐ Source data is mounted into the container through a read-only volume, not baked into the Docker image.
- ☐ The dashboard handles all three states correctly; error messages do not print the connection string.
- ☐ The Deployment Guide has a section with the schedule (one line per schedule) and how to backfill a failed day.
- ☐ No connection string or password in the repo; there is a .env.example.

PART C — AI track: Packaging the model as a prediction service

C1. A prediction service with FastAPI — complete sample code

A trained model is just a file. To make it part of the system you wrap it in a service with a clear interface. FastAPI fits because it validates input automatically (pydantic, returning 422 on invalid input) and generates API documentation at /docs.

src/serve/app.py — prediction service, loads the model ONCE at start-up

```
import os, time, logging, joblib
import numpy as np
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from src.features.text import lam_sach

log = logging.getLogger('serve')
MODEL_PATH = os.environ.get('MODEL_PATH', 'artifacts/model_v1.joblib')
NGUONG_TIN_CAY = float(os.environ.get('NGUONG_TIN_CAY', '0.45'))

trang_thai = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the model ONCE at start-up, not on every request
    goi = joblib.load(MODEL_PATH)
    trang_thai['vectorizer']  = goi['vectorizer']
    trang_thai['model']       = goi['model']
    trang_thai['nhan_labels'] = goi['nhan_labels']
    trang_thai['phien_ban']   = goi.get('phien_ban_du_lieu', 'khong_ro')
    log.info('đã nạp mô hình từ %s', MODEL_PATH)
    yield
    trang_thai.clear()

app = FastAPI(title='Phân loại sự cố – Mekong Mobile', version='1.0',
              lifespan=lifespan)

class YeuCau(BaseModel):
    mo_ta: str = Field(min_length=5, max_length=2000,
                       description='Mô tả sự cố do nhân viên tiếp nhận nhập')

class NhanDuDoan(BaseModel):
    nhan: str
    do_tin_cay: float

class PhanHoi(BaseModel):
    nhan_goi_y: str | None       # None when confidence is too low
    do_tin_cay: float
    top_3: list[NhanDuDoan]
    can_nguoi_xem_lai: bool
    phien_ban_mo_hinh: str

@app.get('/health')
def health():
    """Lets docker-compose and operators know the service is alive."""
    if 'model' not in trang_thai:
        raise HTTPException(503, 'Mô hình chưa nạp xong')
    return {'status': 'ok', 'phien_ban_mo_hinh': trang_thai['phien_ban']}

@app.post('/du-doan', response_model=PhanHoi)
def du_doan(req: YeuCau):
    t0 = time.perf_counter()
    x = trang_thai['vectorizer'].transform([lam_sach(req.mo_ta)])

    if hasattr(trang_thai['model'], 'predict_proba'):
        xac_suat = trang_thai['model'].predict_proba(x)[0]
    else:
        # LinearSVC has no predict_proba - use normalised decision scores
        d = trang_thai['model'].decision_function(x)[0]
        e = np.exp(d - d.max()); xac_suat = e / e.sum()

    thu_tu = xac_suat.argsort()[::-1]
    top3 = [NhanDuDoan(nhan=trang_thai['nhan_labels'][i],
                       do_tin_cay=round(float(xac_suat[i]), 4))
            for i in thu_tu[:3]]

    du_tin_cay = top3[0].do_tin_cay >= NGUONG_TIN_CAY
    log.info('dự đoán xong: tin cậy=%.2f, %.1f ms', top3[0].do_tin_cay,
             (time.perf_counter() - t0) * 1000)      # do NOT log the raw description
    return PhanHoi(
        nhan_goi_y=top3[0].nhan if du_tin_cay else None,
```

```
        do_tin_cay=top3[0].do_tin_cay,
        top_3=top3,
        can_nguoi_xem_lai=not du_tin_cay,
        phien_ban_mo_hinh=trang_thai['phien_ban'],
    )
```

Five design decisions in the service above — each one answers a standard question

1. Load the model once at start-up, not on every request. Loading takes hundreds of milliseconds; reloading on every call turns a fast service into one a hundred times slower.

2. Return the top 3, not a single label. Intake staff need to see the closest options to decide for themselves. A single label forces them to accept or reject it, with nothing in between.

3. A confidence threshold and a "needs human review" flag. When the model is unsure, it says so. This is where engineering meets ethics: the system must not pretend to be certain.

4. Return the model version in every response. Three months later, when someone reports "the system suggested the wrong thing", you need to know which model was running. This is the TRACEABLE principle.

5. A separate /health endpoint. So docker-compose knows when the service is ready, and operators can check it quickly without sending a real prediction request.

C2. A Dockerfile for the model service — does the model go into the image?

Option	How	Advantages	Disadvantages
Model INSIDE the image	COPY artifacts/model_v1.joblib into the image at build time	Runs immediately with no external dependency; best fit for Assignment 3 because the package is self-contained	Large image; changing the model means rebuilding; the model must exist before building
Model mounted through a volume	Mount the artifacts folder into the container at run time	Changing the model needs no rebuild	Not self-contained; others need the model file to run it — fails the clean-machine test
Model downloaded from storage at start-up	The service fetches the model from an address at start-up	Manages many model versions	Needs storage and network; belongs to Specialized Graduation Topic 2

serve/Dockerfile — option 1, suitable for Assignment 3

```
FROM python:3.12-slim AS runtime
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

# Install dependencies first to use the cache
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Run as a non-privileged user
RUN useradd -m -u 10001 app

COPY src ./src
COPY artifacts/model_v1.joblib ./artifacts/model_v1.joblib

USER app
ENV MODEL_PATH=artifacts/model_v1.joblib NGUONG_TIN_CAY=0.45
EXPOSE 8080

HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=6 \
  CMD python -c "import urllib.request,sys; \
    sys.exit(0 if urllib.request.urlopen('http://localhost:8080/health').status==200 else 1)"

CMD ["uvicorn", "src.serve.app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Three things to do before building an image with the model inside

Re-run the training script to produce the latest model — the image will contain exactly the file present at build time.

Check library versions: a model saved with scikit-learn 1.5.2 may fail to load, or behave differently, in an image with 1.6. requirements.txt must pin EXACT versions with ==.

Write in the README the command that regenerates the model before building the image, for example: python -m src.models.train --config configs/train.yaml && docker compose build. The grader must be able to do both steps.

C3. Wiring the model service into the system

docker-compose.yml — the application calls the model service (db as in A2)

```
services:

  model:
    build: ./serve
    environment:
      NGUONG_TIN_CAY: ${NGUONG_TIN_CAY:-0.45}
      LOG_LEVEL:      info
    ports:
      - "${MODEL_PORT:-8080}:8080"

  api:
    build: ./backend
    environment:
      # Call by SERVICE NAME on the compose network, not localhost
      MODEL_URL:        http://model:8080
      MODEL_TIMEOUT_MS: "1500"
      DATABASE_URL:     postgres://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    depends_on:
      db:
        condition: service_healthy    # the DB is ESSENTIAL → wait until healthy
      model:
        condition: service_started    # the model is AUXILIARY → must not block api start-up
    ports:
      - "${API_PORT:-8000}:8000"
```

Calling the model service with failure handling — where Assignment 2 marks are earned

```
async function goiYNhomLoi(moTa, requestId) {
  const ctrl = new AbortController();
  const hetHan = setTimeout(() => ctrl.abort(),
                            Number(process.env.MODEL_TIMEOUT_MS || 1500));
  try {
    const res = await fetch(`${process.env.MODEL_URL}/du-doan`, {
      method: 'POST', signal: ctrl.signal,
      headers: { 'Content-Type': 'application/json', 'X-Request-Id': requestId },
      body: JSON.stringify({ mo_ta: moTa }),
    });
    if (!res.ok) throw new Error(`model trả về ${res.status}`);
    const kq = await res.json();
    return { nguon: 'mo_hinh', ...kq };
  } catch (e) {
    // GRACEFUL DEGRADATION: if the model is down the system STILL RUNS,
    // only the suggestion is lost. Staff pick the issue group by hand as before.
    logger.warn('không gọi được dịch vụ mô hình, chuyển sang chọn tay',
                { requestId, ly_do: e.message });
    return { nguon: 'chon_tay', nhan_goi_y: null,
             can_nguoi_xem_lai: true };
  } finally {
    clearTimeout(hetHan);
  }
}
```

Why the failure handling above is the most important part of C3

The model service is a component that can die: out of memory, a model that fails to load, a slow internal network. This is normal, not a rare case.

Without handling: when the model dies, the whole ticket-intake function dies with it. An auxiliary feature brings down the core business process — a serious design flaw.

With correct handling (graceful degradation): when the model dies, staff choose the issue group by hand as they did before AI. The system still runs; only the convenience is lost.

A maximum wait time is mandatory. Without a timeout, a hanging model service makes every intake request hang too, and the system collapses when connections run out.

Consistent with compose: the api only waits for the model to be service_started, not healthy. If the api had to wait for a healthy model, a broken model would stop the api from starting — contradicting graceful degradation itself.

This is the type of question in question 3 of the final exam: "the system fails after deployment; analyse the causes and propose solutions".

C4. Self-check test [AI]

Run all six steps before the Session 12 workshop

```
# 1. Re-train the model, then build the images
python -m src.models.train --config configs/train.yaml
docker compose down -v && docker compose up -d --build

# 2. Wait for the model service to be ready
docker compose ps                           # model must be "healthy"
curl -s http://localhost:8080/health        # {"status":"ok","phien_ban_mo_hinh":...}

# 3. Call prediction with a REAL description from the data
curl -s -X POST http://localhost:8080/du-doan \
  -H 'Content-Type: application/json' \
  -d '{"mo_ta":"Máy sạc không vào điện, cắm sạc không lên"}' | python -m json.tool

# 4. Call with INVALID data - must return 422 with a clear message
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/du-doan \
  -H 'Content-Type: application/json' -d '{"mo_ta":"abc"}'      # expect 422

# 5. TEST GRACEFUL DEGRADATION - the most important step
docker compose stop model
#    Now call the ticket-intake function through the main API:
curl -s -X POST http://localhost:8000/api/tickets -H 'Content-Type: application/json' \
  -d '{"serialNo":"SN10293A","customerId":1024,"mucDo":"CAO","moTaLoi":"Máy sạc không vào điện"}'
#    It MUST still create the ticket, just without a suggested label.
#    If it returns 500 -> failure is not handled; fix it before submitting.
docker compose start model

# 6. Read the model service logs
docker compose logs model --tail 20
```

- ☐ The model is loaded once at start-up, not on every request.
- ☐ There is a /health endpoint and it returns the model version.
- ☐ The prediction response has the top 3 labels, the confidence, and the needs-review flag.
- ☐ Every response returns the model version.
- ☐ The confidence threshold comes from an environment variable, not hard-coded.
- ☐ Invalid input returns 422 with a message saying which field is wrong.
- ☐ You have stopped the model service and the core business process STILL works (graceful degradation).
- ☐ The call to the model service has a maximum wait time.
- ☐ requirements.txt pins exact versions; the model loads inside the Docker image.
- ☐ The README states two steps clearly: regenerate the model, then build the image.

PART D — Common to all three tracks: Bug Log, logging and error handling

This part implements three theory topics of Session 11 as concrete files. The SE track already has a logger and an error middleware in Node.js (Part A3 and the Session 11 slides); the DA and AI tracks use the Python code below.

D1. The Bug Log — write it as soon as a bug is fixed, do not wait for Assignment 3

Every time you go through the four steps reproduce → isolate → fix the cause → verify, you have a Bug Log entry ready. Assignment 3 requires at least three entries, at least one of them Critical or Major, each with all seven lines and a commit link that opens.

docs/bug-log.md — one sample entry with the seven lines Assignment 3 requires

```
## BUG-004 · Major — Ticket creation fails for customers without a name

| Line                    | Content |
|-------------------------|---------|
| 1. ID and severity      | BUG-004 · Major (a group of customers cannot get a ticket) |
| 2. Detected by          | TC-07 "create a ticket for a customer imported from legacy data" |
| 3. Symptom              | POST /api/tickets returns 500; the UI shows "Có lỗi xảy ra" |
| 4. How to reproduce     | curl -X POST localhost:8000/api/tickets -d @tests/fixtures/bug004.json |
| 5. Cause                | toTicketView() calls trim() on full_name = null; customers_raw.csv |
|                         | has 37 records without a name and no check rule ran on load |
| 6. Fix                  | The mapper uses "(không rõ)" when the source is empty (as the contract |
|                         | says); new rule CL6 "missing name" → such rows go to the reject table |
| 7. Commit and verify    | https://github.com/<account>/<repo>/commit/a1b2c3d |
|                         | test "tạo phiếu được khi khách thiếu họ tên" FAILS before, PASSES after |
```

Three mistakes that make a Bug Log entry not count

Line 5 states the symptom instead of the cause. "The API returns 500" is a symptom. Keep asking "why" until you reach the first place where a contract is violated — that is the cause.

A dead commit link, or one pointing to an unrelated commit. The lecturer clicks every link. Fix commits should be small with a clear message: fix(ticket): xử lý khách thiếu họ tên khi tạo phiếu.

No verifying test. Step 4 produces a regression test; it must FAIL before the fix and PASS after it. Describing a bug you did not actually fix is an academic-integrity issue.

D2. Logging in Python — configure once, write JSON to stdout

src/log_config.py — [DA/AI] configure logging ONCE, write JSON to stdout

```
import json, logging, os, sys

class DinhDangJson(logging.Formatter):
    """Each log line is a JSON object: time, level, source, message, context."""
    TRUONG_NGU_CANH = ('ma_lo', 'request_id', 'so_dong', 'ma_quy_tac')

    def format(self, r):
        d = {'thoi_diem': self.formatTime(r, '%Y-%m-%dT%H:%M:%S%z'),
             'muc': r.levelname, 'nguon': r.name, 'thong_diep': r.getMessage()}
        for k in self.TRUONG_NGU_CANH:              # only the declared context fields
            if hasattr(r, k): d[k] = getattr(r, k)
        if r.exc_info: d['loi'] = self.formatException(r.exc_info)
        return json.dumps(d, ensure_ascii=False)

def cau_hinh_log():
    h = logging.StreamHandler(sys.stdout)           # stdout, NOT a file inside the container
    h.setFormatter(DinhDangJson())
    logging.basicConfig(level=os.getenv('LOG_LEVEL', 'INFO'), handlers=[h], force=True)

# Use at the entry point (src/pipeline.py or src/serve/app.py):
#   cau_hinh_log()
#   log = logging.getLogger('pipeline.transform')
#   log.warning('bỏ qua dòng không quy đổi được ngày',
#               extra={'ma_lo': ma_lo, 'so_dong': 14})
```

Three rules for logging in Python

Call cau_hinh_log() exactly once at the program's entry point. Modules only call logging.getLogger(__name__) — never basicConfig in every file.

No print in submitted code. print has no level, no timestamp and cannot be switched off. Turn debugging prints into log.debug(...) and set LOG_LEVEL=INFO.

log.exception only where the error is HANDLED (pipeline.py, the API error handler). Do not log and re-raise in every function — one incident would become three ERROR lines.

D3. Cross-layer error handling for Python APIs — one single handler

The same principle as the Express middleware in the Session 11 slides: the business layer raises errors with a code; one single handler translates them into HTTP status codes using the { error, message } format from Session 9 and logs exactly once. Call dang_ky_xu_ly_loi(app) right after creating the app in src/serve/app.py.

src/serve/errors.py — [AI, and any Python API] business errors and a single error handler

```
import logging, uuid
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

log = logging.getLogger('serve.errors')

class LoiNghiepVu(Exception):
    """A business-rule violation: returns 4xx, NOT a system incident."""
    def __init__(self, ma: str, thong_diep: str, http_status: int = 409):
        super().__init__(thong_diep)
        self.ma, self.thong_diep, self.http_status = ma, thong_diep, http_status

def dang_ky_xu_ly_loi(app: FastAPI):
    @app.exception_handler(LoiNghiepVu)
    async def _nghiep_vu(req: Request, e: LoiNghiepVu):
        log.info('vi phạm quy tắc %s', e.ma)                        # INFO, not ERROR
        return JSONResponse(status_code=e.http_status,
                            content={'error': e.ma, 'message': e.thong_diep})

    @app.exception_handler(Exception)
    async def _he_thong(req: Request, e: Exception):
        ma_tra_cuu = req.headers.get('x-request-id') or uuid.uuid4().hex[:8]
        log.exception('lỗi hệ thống', extra={'request_id': ma_tra_cuu})   # logged ONCE
        return JSONResponse(status_code=500, content={
            'error': 'LOI_HE_THONG',
            'message': f'Hệ thống đang gặp sự cố. Mã tra cứu: {ma_tra_cuu}'})
        # NEVER return str(e) or the traceback to the caller
```

Layer	Responsibility when an error occurs	Must not
Data (repository, extract, model loading)	Wrap library errors into application errors, keeping the original cause (cause / raise ... from e)	Return HTTP codes; write logs; swallow the error and return None
Business (service, transform, quality)	Check QT-xx rules; on violation raise LoiNghiepVu with a CODE; do not catch infrastructure errors it cannot handle	Know about HTTP (req, res, status); catch a generic Exception
Boundary (middleware, exception_handler, pipeline.py)	Translate the error → HTTP status or exit code; log ONCE with the requestId / batch ID	Send stack traces, SQL or connection strings to the user
UI / dashboard / caller	Branch on error; show message: what happened · why · what to do next; a reference code for 5xx errors	Show a technical error dump; a blank screen

D4. Common checklist before the Session 12 workshop

- ☐ The lock file is committed (package-lock.json, or requirements.txt with == generated from a clean venv); the README states the runtime version.
- ☐ docs/bug-log.md has at least one seven-line entry for a bug you actually fixed.
- ☐ grep -rnE "catch *\([^)]*\) *\{ *\}|except *: *pass" src/ returns nothing — no swallowed errors.
- ☐ No debugging print/console.log left in src/; logs have a level, a timestamp and context.
- ☐ Logs contain no passwords, tokens, connection strings or customer phone numbers.
- ☐ User errors return 4xx with a clear message; system errors return 5xx with a generic message and a reference code.
- ☐ You have run your track's self-check test from an empty folder and noted the first failing step (if any).

Quick comparison of the three tracks — a self-check reference table

Criterion	SE track	DA track	AI track
Services in compose	db · api · web	warehouse · pipeline · dashboard	db · model · api (· web)
Which service runs once and exits	None	pipeline	None (but training runs outside compose)
Where start-up order matters	api waits for db healthy	pipeline and dashboard wait for warehouse healthy	api waits for db healthy; only for model started
How large data is handled	Small seed in the repo	Source data on a read-only volume, small sample in the repo	Model copied into the image; training data not in the image
Where logs and errors are handled	JSON logger + single error middleware	JSON logs + pipeline.py decides to stop, exit codes 0/1/2	JSON logs + single exception_handler
Where graceful degradation happens	Front end loses the API → error state with a retry button	Dashboard loses the warehouse → clear error message	API loses the model service → staff choose by hand, the process still runs
The track's most serious mistake	depends_on without the healthy condition	Loading data before quality checks	A dead model service brings down the core process
The final test, identical for all three	down -v then up --build from an empty folder	down -v then up --build from an empty folder	down -v then up --build from an empty folder

Three things to do this week, before the Session 12 workshop (Assignment 2 submission)

1. Run your track's self-check test from start to finish and write down which step fails. The first failing step is your number-one task in Session 12.

2. Test GRACEFUL DEGRADATION. Stop an auxiliary service and see whether the system still runs. If it collapses, fix it now — this is step 5 of the Assignment 2 grading procedure and the type of question in exam question 3.

3. Review secrets. Run git log --all --oneline -- .env — if it prints anything, .env was committed at some point: change every related password and key immediately and record that in your documentation. Deleting it in a later commit is not enough, because the secret stays in the git history.